Spaces, Duals, Morphisms, and `BlockTensor`

August 4, 2026

Abstract

This note refines the local tensor model in *Planar Tensor Networks, Braiding, and Interface Invariants*. The planar paper established that a symmetry-aware tensor is a morphism with an ordered boundary and that global contraction semantics belong to a checked planar diagram or contraction plan. The refinement here separates two distinctions that the earlier Space/CoSpace spelling conflated: categorical object duality, Space/DualSpace, and morphism side, Domain/Codomain. Moving a boundary factor from one side of a morphism to the other is consequently an explicit wire-bending operation that toggles object duality. This model applies unchanged to the first bosonic U(1) BlockTensor, where it reduces to signed charge bookkeeping, and to later non-Abelian or braided implementations, where it carries fusion, pivotal, and braid data.

Contents

1 Relationship to the Planar-Network Design

The earlier planar-network note supplies the global architecture:

1. a tensor stores a local ordered domain and codomain;
2. a planar network stores the embedding and external boundary;
3. a contraction plan inserts and lowers the required local compositions, recouplings, braids, and traces.

Those conclusions remain unchanged. This note corrects and completes the local boundary vocabulary. The earlier term CoSpace attempted to encode "the other direction" in one type. There are, however, two independent questions:

1. Is the categorical object X or its dual $X^\vee$?
2. Does this occurrence appear in the domain or codomain?

The answers must not be collapsed. A domain may contain either X or $X^\vee$, and so may a codomain. The durable vocabulary is therefore Space/DualSpace for object duality and Domain/Codomain for morphism side.

2 Five Pieces of Boundary Data

A symmetry-aware tensor boundary contains five different forms of information. Keeping them separate prevents structural metadata from silently acquiring mathematical semantics.

2.1 Symmetry or Tensor Category

The symmetry context supplies the category $\mathcal{C}$. It owns:

- sector fusion and the tensor unit;
- object duality;
- invariant-channel multiplicities;
- evaluation and coevaluation maps;
- eventually associators, pivotal data, twists, and braiding.

For bosonic U(1), most of this structure is trivial. Fusion is charge addition, the tensor unit has charge zero, duality negates charge, and every allowed invariant channel has multiplicity one. This is a specialization of the interface, not a different interface.

2.2 Structural Decomposition

A decomposition records which sectors and degeneracy dimensions a space contains. The currently implemented Uni20 types provide useful structural forms:

Form	Structural data	Typical role
`BlockSpace`	(q, d_q) sectors	MPS virtual bond
`QNumList`	ordered local charges	physical site
`QNum`	one irrep label	irreducible operator
dense extent	no symmetry sectors	bundled dense axis

Structural equality is not semantic identity. Two MPS bonds can have the same U(1) sectors and degeneracies while belonging to different cuts of the chain.

2.3 Identity-Bearing Space

A Space is an immutable semantic object over one decomposition. It carries a SpaceId.

- Copying a space preserves its identity.
- Constructing an isomorphic space creates a distinct identity.
- Exact composition normally requires the same identity.
- Composition through distinct isomorphic spaces requires an explicit isomorphism.
- A display label is not the identity.

The implementation should expose the conceptual distinction

$$\text{same\_space}(X, Y) \qquad \text{versus} \qquad \text{isomorphic\_space}(X, Y).$$

The first checks identity; the second checks compatible categorical and structural data.

A process-local address is not a sufficient SpaceId. MPI construction and distributed decompositions require an identity that can be serialized or deterministically reproduced on every participating MPI rank.

2.4 Object Duality

For a space X, $X^\vee$ is represented by $\mathsf{DualSpace}\langle X\rangle$. It retains the identity of its primal space:

$$(X^\vee)^\vee \cong X, \qquad \text{primal\_id}(X^\vee) = \text{space\_id}(X).$$

The dual is not a newly allocated, unrelated space whose decomposition happens to contain dual sectors. For U(1), its sector q is represented by $-q$. In a richer category the symmetry context also supplies the evaluation, coevaluation, and normalization data required to use the dual.

2.5 Morphism Side and Leg Occurrence

A tensor is a morphism

$$A : X_0 \otimes \cdots \otimes X_{m-1} \longrightarrow Y_0 \otimes \cdots \otimes Y_{n-1}.$$

Its local boundary is the ordered pair

$$\mathsf{Domain}[X_0, \ldots, X_{m-1}], \qquad \mathsf{Codomain}[Y_0, \ldots, Y_{n-1}].$$

Every entry is one leg occurrence. Several occurrences may refer to the same space, just as an identity map has the same space in its domain and codomain. Occurrence identity, position, display label, space identity, and object duality are distinct.

3 The All-Out Boundary

Fusion and block legality are most naturally expressed by orienting every boundary strand outward. Under this convention,

$$\mathsf{Codomain}[X] \longmapsto X, \qquad \mathsf{Domain}[X] \longmapsto X^\vee.$$

If an entry already names a dual object, the two dualities cancel. Effective outward duality is therefore the exclusive-or of the domain-side flag and the explicit dual-object flag.

For the morphism above, the all-out boundary is

$$X_{m-1}^{\vee} \otimes \cdots \otimes X_0^{\vee} \otimes Y_0 \otimes \cdots \otimes Y_{n-1},$$

where the reversal of the dualized product records

$$(X_0 \otimes \cdots \otimes X_{m-1})^{\vee} \cong X_{m-1}^{\vee} \otimes \cdots \otimes X_0^{\vee}.$$

An implementation may use a different canonical cyclic starting point, but it must choose and document one order and preserve it through every operation.

A populated block corresponds to an invariant channel

$$\mathrm{Hom}_{\mathcal{C}}\left(\mathbf{1}, X_{m-1}^{\vee} \otimes \cdots \otimes X_0^{\vee} \otimes Y_0 \otimes \cdots \otimes Y_{n-1}\right).$$

The generic selection query is consequently a category operation over oriented sectors, not a hard-coded arithmetic predicate.

3.1 U(1) Selection Rule

For bosonic U(1), the invariant-channel query reduces to

$$\sum_{j=0}^{n-1} q(Y_j) - \sum_{i=0}^{m-1} q(X_i) = 0.$$

For an MPS site

$$A : L \otimes P \longrightarrow R,$$

this is

$$q_R = q_L + q_P.$$

For an MPO site

$$W : L \otimes K \longrightarrow R \otimes B,$$

it becomes

$$q_L + q_K = q_R + q_B.$$

These are instances of one oriented-boundary rule.

4 Repartition Is an Explicit Bend

Changing which side of a morphism contains a leg is not a relabel or a bit flip. It is a rigid-category isomorphism built from evaluation or coevaluation:

$$\mathrm{Hom}_{\mathcal{C}}(A \otimes X, B) \cong \mathrm{Hom}_{\mathcal{C}}(A, B \otimes X^{\vee}),$$

$$\mathrm{Hom}_{\mathcal{C}}(A, B \otimes X) \cong \mathrm{Hom}_{\mathcal{C}}(A \otimes X^{\vee}, B).$$

The operation is provisionally named `repartition`. Moving an occurrence across the boundary toggles its explicit Space/DualSpace status so that the all-out boundary is preserved.

Moving several factors also exposes the contravariant ordering rule:

$$(X \otimes Y)^{\vee} \cong Y^{\vee} \otimes X^{\vee}.$$

The repartition API must therefore specify the resulting ordered boundary, not only a new domain size.

In bosonic U(1), a bend can normally be implemented as checked metadata and an axis-order view. In a nontrivial pivotal category it may carry normalization maps or twists. It must always route through the category hook even when that hook returns the identity.

4.1 Operations That Must Remain Distinct

- `relabel`: changes diagnostic metadata only.
- `conj`: changes numerical value semantics while preserving the morphism boundary.
- `repartition`: bends selected strands and toggles object duality.
- `adjoint`: reverses the morphism and applies the category's adjoint and recoupling rules.
- `braid`: exchanges strands using the category's braiding.

These operations can have similarly shaped dense-array implementations in the trivial case. That does not make their contracts interchangeable.

5 Contraction as Tensor Product and Composition

Full contraction is composition. Given

$$A : X \longrightarrow Y, \qquad B : Y \longrightarrow Z,$$

the result is

$$B \circ A : X \longrightarrow Z.$$

Composition checks ordered boundary compatibility and space identity, not only dimensions or sectors.

5.1 Partial Contraction

Partial contraction is tensor product with identities followed by composition. Suppose

$$A : L \longrightarrow M \otimes X, \qquad B : X \otimes R \longrightarrow N.$$

Construct conceptually

$$A \otimes \mathrm{id}_R : L \otimes R \longrightarrow M \otimes X \otimes R,$$
$$\mathrm{id}_M \otimes B : M \otimes X \otimes R \longrightarrow M \otimes N.$$

Their composition is the requested contraction:

$$(\mathrm{id}_M \otimes B) \circ (A \otimes \mathrm{id}_R) : L \otimes R \longrightarrow M \otimes N.$$

Uni20 does not need to materialize either identity tensor. The checked planner proves the categorical expression, determines the external boundary, and lowers directly to block worklists.

5.2 General Planar Networks

A planar network can be sliced into layers of tensor products followed by compositions. The general planner may additionally require:

- associators and F-moves;
- repartition bends;
- symmetric permutations or explicit R-moves;
- categorical traces for closed strands.

The local BlockTensor owns its ordered morphism boundary. The network owns cyclic orders, global adjacency, and crossings. A local tensor cannot decide whether a proposed pairing is planar without that network context.

6 Why Bosonic Abelian Tensors Use the Full Model

It is tempting to implement U(1) as a flat list of signed legs and add the categorical model later. That would make the first implementation smaller only by postponing its central invariants.

The full model immediately provides:

- exact bond-space identity checks;
- explicit input and output boundaries;
- one contraction language for U(1) and richer categories;
- correct creation and sharing of decomposition spaces;
- a category-shaped selection-rule interface.

For bosonic U(1), it should compile to the inexpensive representation:

- multiplicity-free fusion;
- trivial associators and symmetric braiding;
- charge negation for object duality;
- an empty coupling descriptor;
- signed charge sums for block legality;
- metadata-only canonical bends and permutations.

A convenience contraction API may insert the unique canonical bend or permutation in this symmetric setting. It still produces a checked plan. A braided category must reject an expression that leaves a crossing unspecified.

7 Consequences for `BlockTensor`

The block-sparse container should be parameterized by a morphism boundary rather than a flat list of direction-prefixed leg types. Illustrative C++ spelling is:

```
template <
    class T,
    class MorphismSpec,
    class Storage = HostOnly,
    class Coupling = Trivial>
class BlockTensor;

using MpsSiteSpec = MorphismSpec<
    Domain<BlockSpace, LocalSpace>,
    Codomain<BlockSpace>>;

using MpoSiteSpec = MorphismSpec<
    Domain<BlockSpace, LocalSpace>,
    Codomain<BlockSpace, LocalSpace>>;
```

The exact class names remain design sketches. The required split is:

- the static spec fixes order and decomposition kinds;
- tensor values carry the actual identity-bearing spaces;
- each boundary occurrence refers to one of those spaces or its dual;
- storage and placement policies do not change the boundary.

7.1 Block Keys and Coupling Data

A block key is an extensible structure

$$\mathsf{BlockKey} = (\text{sector index per boundary occurrence}, \mathsf{CouplingDescriptor}).$$

For multiplicity-free U(1), the coupling descriptor is empty. For non-Abelian fusion it identifies an invariant channel or fusion basis. The container must not assume there is exactly one block for every legal flat sector tuple.

Block construction calls a category-shaped query such as

```
symmetry.invariant_channels(oriented_sectors)
```

and constructs only the channels it returns.

7.2 Storage Orthogonality

The same boundary and block key can be stored as individual host allocations, offsets into a coalesced buffer, CUDA-resident blocks, or MPI-distributed blocks. A storage policy chooses ownership, allocation, and placement. It must preserve:

- every SpaceId;
- domain and codomain order;
- explicit object duality;
- sector indices and coupling descriptors;
- the global block identity used by distributed worklists.

Storage movement is not a symmetry transformation.

8 Decompositions Create New Spaces

An SVD is not only a dense matrix factorization. It creates a new intermediate space. For

$$A : X \longrightarrow Y,$$

one coherent convention is

$$V : X \longrightarrow B, \qquad S : B \longrightarrow B, \qquad U : B \longrightarrow Y,$$

$$A = U \circ S \circ V.$$

The factor naming may follow the final linalg API, but the boundary contract is fixed:

1. B receives one new SpaceId.
2. Every factor occurrence of the decomposition bond shares that identity.
3. Truncation determines the structural decomposition of B.
4. An independently created isomorphic space is not silently substituted for B.

QR, LQ, polar decomposition, and DMRG bond truncation obey the same rule. This is a direct practical benefit of semantic space identity in the first U(1) implementation.

9 Operation Vocabulary

Operation	Contract
`tensor_product(A,B)`	ordered monoidal product
`compose(B,A)`	exact morphism composition
`repartition(A,...)`	boundary bend with duality toggle
`contract(A,B,pairs)`	checked partial-contraction planner
`trace(A,pair)`	categorical trace
`braid(A,...)`	explicit strand exchanges
`relabel(A,...)`	diagnostic metadata only

The planner may lower these operations to axis permutations, dense GEMMs, elementwise kernels, or communication. Those lower operations do not decide space compatibility, fusion legality, or planarity.

10 Initial U(1) Implementation Sequence

The first host-only U(1) BlockTensor does not require general F- or R-symbol machinery. It should nevertheless establish the following sequence:

1. Add immutable identity-bearing spaces over the implemented structural decompositions.
2. Represent ordered domain and codomain boundaries.
3. Derive block legality from the all-out boundary.
4. Keep the block key extensible with an empty initial coupling descriptor.
5. Check exact space identity in composition and contraction.
6. Implement the cheap U(1) form of repartition.
7. Make SVD and truncation create and share one new bond space.
8. Carry boundary and space metadata through async worklists, reduced-block kernels, CUDA lowering, and later MPI placement.

There is no implicit dense fallback for a symmetry-typed operation. An explicitly named dense projection may exist for diagnostics, but it leaves the U(1) execution path and cannot feed data back into the symmetry-aware state.

11 Interface Invariants

The following invariants refine the planar-network invariants and should remain testable:

1. Every tensor has an ordered domain and ordered codomain.
2. Object duality is independent of morphism side.
3. A dual object retains its primal space identity.
4. Repartition toggles explicit duality and respects order reversal.
5. Composition checks space identity and boundary compatibility.
6. Isomorphic but distinct spaces require an explicit isomorphism.
7. Labels and diagram coordinates never change legality.
8. Block selection is an invariant-channel query over the oriented boundary.
9. Global planarity belongs to the network or planner.
10. Crossings, recouplings, and traces are explicit categorical operations.
11. Storage movement preserves all boundary, sector, and coupling metadata.

11.1 Required Tests

The initial test suite should include:

- two isomorphic equal-dimensional spaces with different identities that cannot be composed accidentally;
- U(1) MPS and MPO block selection using the signed boundary equations above;
- round-trip repartition returning the original boundary and data;
- multiple-leg repartition with the required order reversal;
- SVD factors sharing exactly one new bond identity;
- relabeling that has no effect on contraction legality;
- worklist generation that retains every logical block key and space identity.

12 Conclusion

The local tensor boundary has two independent categorical axes: Space/DualSpace and Domain/Codomain. Treating them as one "direction" type obscures wire bending, makes general contraction harder to state, and loses information needed by non-Abelian and braided categories.

The practical rule is:

> Select blocks from an oriented morphism boundary, compose through identity-bearing spaces, bend wires explicitly, and leave global planarity to the contraction planner.

For bosonic U(1), these rules reduce to cheap identity checks and signed charge bookkeeping. They are therefore suitable for the first host-only DMRG implementation while preserving the structure required for CUDA, MPI, non-Abelian fusion, and braiding.